

Scope of Work — Activities Page (Mbiufun Platform)

The Activities Page will be transformed from a simple content feed into a full **event creation, event participation, and social interaction system**. This Scope of Work outlines all required functional, frontend, backend, and integration responsibilities.

1. Overview / Purpose

The Activities Page will allow users to:

- Create real-world activities or events
- Join existing activities
- View activity details (date, location, description, participants)
- Interact through MbiuHorn likes, comments, and shares
- See upcoming events in a clean, user-friendly interface

This page becomes the **action hub** of Mbiufun — connecting users through real-world plans.

2. Core Features

The Activities Page will include the following main components:

A. Create Activity / Event

Users can create an event with:

- Title
- Description
- Date
- Start & end time
- Location
- Hobby category
- Optional image
- Privacy: Public / Friends Only / Invite Only
- Maximum participants

B. Join Activity

- Join button
- Cancel join
- Updated participant count
- Visual confirmation

C. Activity Details

- Full event info
- List of participants
- Activity creator
- Time & location display
- Optional map preview (if implemented later)

D. Engage with Activity

- MbiuHorn (like)
- Comment system
- Share (Facebook, Instagram, link)

E. Listing of Activities

- “Upcoming Activities” feed
 - Sorted by date
 - Show distance / relevance (optional future upgrade)
-

3. Frontend Scope (Angular)

Components to Build

1. `activities-list.component`
2. `activities-create.component`
3. `activities-details.component`
4. `activities-join.component`
5. `activities-comments.component`
6. `mbiuhorn-like.component`
7. `activities-share.component`
8. `activities-card.component`

Frontend Tasks (Step-by-Step)

A. Activities List

1. Build list layout for all upcoming activities
2. Implement scroll & pagination
3. Display date, location, participants, hobby type
4. Add "Join / Joined" button
5. Add horn, comment count, share icons

B. Activity Creation Form

1. Form UI: title, description, date, time, location
2. Add upload image field
3. Add hobby picker
4. Add privacy selector
5. Add form validation & error messages
6. Build success modal

C. Activity Details Screen

1. Clean layout with hero banner
2. Event info: date, description, location
3. Participants list view
4. Comments section
5. Join / Cancel button state management
6. Share button popup

D. Horn & Comment Interactions

1. Horn animation on click
2. Comments UI with timestamp + user
3. Reply (optional)
4. UI logic for preventing spam

E. Share Integration

1. Create share modal
 2. Add OS-native share API
 3. Add fallback for copy-link
-

4. Backend Scope (Django)

Backend Models

1. `Activity`
2. `ActivityParticipant`
3. `ActivityComment`
4. `ActivityHornBlast`

Backend Tasks (Step-by-Step)

A. Activity Management

1. Create Activity model (fields: title, description, date, time, location, hobby, capacity, etc.)

2. Create Activity CRUD endpoints
3. Validate date $\geq$ today
4. Validate capacity

B. Joining System

1. Create participants table
2. API: join event
3. API: cancel join
4. Prevent duplicate joins
5. Enforce max participants

C. Interaction System

1. MbiuHorn (like) endpoint
2. Prevent duplicate horns
3. Create comment API
4. Comment validation + sanitization

D. Activity Feed

1. Paginated endpoint for all upcoming events
2. Sorting by date/time
3. Filtering by hobby (optional)

E. Sharing Endpoint

1. Generate shareable event link
 2. Ensure privacy rules
 3. Optionally return preview data
-

5. Expected Deliverables

- Fully functional Activities page
 - All frontend components implemented and responsive
 - Complete backend API layer with authentication
 - Connected frontend & backend with clean event flows
 - Validated, error-handled forms and interactions
 - Share-ready activity links
-

6. Dependencies / Integrations

- Django authentication & user model
 - Angular routing & state management
 - Optional: Google Maps API for locations
 - OS-native sharing
 - Database: PostgreSQL
-